

Supplementary Material: Agulhas DeepONet / CNN Double-Penalty Study

This document holds the detailed, chronological account of methodological corrections and intermediate verification steps referenced by the main text’s Sec. 4.2 and Sec. 4.4. It exists in response to external review noting that this level of research-process detail — which bug was found when, what was re-run, how confidence was restored — belongs in a supplement rather than the main results narrative. The main text states each correction’s *effect on the reported numbers* concisely and points here for the full account of *how each was found and fixed*.

S1. The four methodological corrections in full (supplements main text Sec. 4.2)

This paper discloses four corrections made during this project, each large enough to have changed a headline number if left uncaught. Every number in the main text already reflects all four; this section is the full account of what each one was and how it was caught, not something still to be adjusted.

Comparison design. An earlier, imperfectly-controlled version of the AMSE-vs-control comparison (every AMSE seed compared against a single, arbitrarily chosen control seed rather than its own seed-matched control) suggested AMSE’s benefit was concentrated in position error specifically, with recall significant in only 5 of 10 seed/weight combinations. Redoing the comparison with each AMSE seed paired against a control trained at the *same* seed — removing control-seed-specific noise from every comparison — produced the fully uniform 10-of-10 result reported in the main text’s Table 6 ($r = 6$, the resolution this fix was originally verified against). Comparison design (which control a given seed is measured against) mattered as much here as sample size did, and a reader auditing the underlying result files would find the inconsistency if we did not flag it ourselves. We have since added a runtime guard to the comparison tool itself (`eddy_stat_test.py`) that refuses to run an *a*-vs-*b* comparison across mismatched seeds unless explicitly overridden, so this specific mistake cannot happen silently again.

An alternative loss, tried first and abandoned. Before AMSE, we tried a differentiable, neighborhood-based auxiliary loss on the SSH field — a soft Fractions Skill Score construction (Roberts & Lean, 2008), in the spatial-verification-informed loss-function tradition established by Lagerquist & Ebert-Uphoff (2022) for thunderstorm prediction. A first, single-seed screen looked promising: a significant recall improvement over a matched control ($p = 0.035$). It did not replicate. Two additional seeds gave a sign flip — one seed significant *against* the treatment at $p < 0.0001$, the opposite direction from the first seed at comparable confidence. We read this as a real, informative negative result illustrating seed-sensitivity risk specific to this class of thresholded, detection-proxy loss, and it directly motivated trying a loss with an *exact* decomposition (AMSE’s Parseval identity) rather than a softened, thresholded approximation of an object-detection signal — the mechanism that ultimately replicated cleanly.

Learning rate. Every full-scale AMSE result was initially trained at $\text{lr} = 3 \times 10^{-4}$ — the DeepONet’s own validation-selected rate, copied into the AMSE training scripts by mistake rather than the CNN’s own established-optimal $\text{lr} = 1 \times 10^{-3}$ (main text Sec. 2.3). This was caught by comparing against the officially confirmed CNN baseline predictions and noticing a discrepancy in grid skill too large to be architecture-identical seed noise, and every full-scale result in this paper was retrained at the corrected rate before being reported: an architecture-appropriate hyperparameter was implicitly assumed to transfer across architectures, and it did not. This exact class of bug was still live in this project’s own SLURM training scripts as of this section originally being written — caught only while building a static regression test in direct response to a formal review’s request for systematic auditing (`test_lr_config_regression.py`), which now checks every CNN-training script’s effective learning-rate default and would fail loudly if this specific mistake recurred.

Eddy-detection library version. Every eddy-tracking number in an earlier version of this paper — the $r = 6$ comparisons the main text now reports as Tables 4 through 7, and the true-eddy counts they were built on — was computed against a contour-detection pipeline that was silently broken. `py-eddy-tracker` (Mason et al., 2014) extracts per-level SSH contours via a matplotlib `QuadContourSet` API (`.collections`) that matplotlib deprecated and behaviorally restructured starting in version 3.8; `py-eddy-tracker`’s own dependency declaration does not pin an upper bound on matplotlib, so our environment had silently picked up a version past the break. The effect was not a crash or a warning surfaced to us at the time — every contour level in a given field silently collapsed to at most one detected path, regardless of how many real, disjoint eddies were actually present, so the true-eddy census this paper originally reported ($n = 1,962$) was itself only a small fraction of what the same underlying SSH field actually contained. We found this only while pursuing an unrelated validation (an independent altimetry-truth comparison, briefly summarized in the main text’s Sec. 3.3), when the true-eddy density implied by the existing pipeline turned out to be implausibly low compared to an independent eddy atlas, and traced it to the root cause by reproducing the exact failure mode on a synthetic test field with known, unambiguous ground truth. The fix (pinning matplotlib to a pre-3.8 version) was verified against that synthetic case before being trusted, and every full-scale table in this paper was regenerated end-to-end under the corrected environment; this paper’s headline conclusions survive the fix, several with a stronger statistical margin than before, but essentially every exact figure changed, and one specific comparative claim (the main text’s Sec. 3.6 polarity finding, from the $r = 6$ comparisons this bug affected) reversed and is reported corrected rather than silently re-derived. The ad hoc synthetic test that found this specific bug has since been formalized into a permanent regression test (`eddy_tracking/test_synthetic_ground_truth.py`, main text Sec. 2.5) with the same design: hand-placed eddies at known positions, run through the actual pipeline, detected count and position checked against what was planted. We confirmed it has real diagnostic power by running it against both matplotlib versions directly: it fails outright (0 of 10 detected) under the pre-fix version and passes (10 of 10) under the fix.

What this section is, and is not. We disclose all four corrections directly rather than silently absorbing them, because a third-party dependency silently drifting past a compatibility boundary, a copy-pasted hyperparameter, and a mismatched-control comparison design are exactly the kind of failures a verification-conscious study should look for, and we would rather a reader learn about them here than discover them independently. In direct response to a formal review’s request for systematic auditing beyond ad hoc, after-the-fact discovery, we have added regression tests for three of these four failure modes — comparison-design pairing, learning-rate configuration, and eddy-detection ground truth — that run without cluster access and are meant to be run before, not only after, a future full-scale result is trusted; we have not built one for the fourth (the abandoned FSS loss), which was correctly identified as unreliable by its own seed-sensitivity check rather

than silently trusted, so there is no comparable silent-failure mode to guard against. None of this substitutes for independent reproduction of this paper’s central tables by a party outside this project, which has not happened (main text Sec. 4.5).

S2. The physics-informed finding in full (supplements main text Sec. 4.4)

Main text Sec. 4.4 states this finding’s result in one paragraph and points here for the complete account: this section holds both key tables, the intermediate weight-sweep investigation, and the mechanistic account of why weight and architecture jointly determine the outcome. This is a secondary, self-contained finding, distinct in mechanism from this paper’s central double-penalty/AMSE contribution (main text Sec. 2.7 explains why physics-informed constraints were tried in the first place, as part of early DeepONet development).

The reversal. Sec. 2.7’s physics-informed null result (divergence-free/geostrophic penalties never help the DeepONet, traced to the training labels’ own residual violation sitting at the physical noise floor) was tested only on the whole-domain DeepONet, the architecture main text Sec. 1/3.1 establishes as the wrong choice for this task. Re-run on the CNN used everywhere else in this paper — the same weight (0.1) that was the DeepONet’s own winning configuration, the same 5 seeds (2026, 7, 42, 123, 2027), the same corrected learning rate (Sec. 2.3/4.2) — the null result does **not** replicate.

Table 1: Geostrophic-consistency penalty ($w=0.1$, the DeepONet’s own winning weight) on the CNN vs. the CNN control, seed-matched, $n=5$.

Seed	Grid skill, physics	Grid skill, control	Diff
2026	0.3195	0.3763	−0.0568
7	0.3123	0.3747	−0.0624
42	0.3195	0.3916	−0.0721
123	0.3097	0.3697	−0.0600
2027	0.3156	0.3860	−0.0704
Mean	−0.0643 ± 0.0066 ($t = -21.66$)		

Every one of the 5 seeds shows the same direction and a comparable magnitude (individual diffs −0.057 to −0.072 — not a seed-noise artifact). Autoregressive rollout is affected far more severely: 20-day mean skill collapses from the CNN control’s -0.09 ± 0.11 (main text Table 1) to -4.82 ± 1.28 under the $w=0.1$ physics penalty, roughly a 50-fold worsening.

An open confound, checked directly rather than left flagged. The weight used in Table 1 (0.1) was selected for the DeepONet, not the CNN — this paper has already caught and disclosed the identical mistake for learning rate (Sec. S1 above: a hyperparameter tuned on one architecture silently assumed to transfer to another), and reporting the CNN’s harm at an untuned weight as “architecture-dependent” without first checking a CNN-appropriate weight would repeat that mistake rather than correct for it. We ran a CNN-specific geostrophic-weight sweep (`cnn_physics_weight_sweep.slurm`, single seed 2026, four weights spanning two orders of magnitude below 0.1) before treating Table 1 as the final word. The result resolves the confound rather than merely flagging it: the harm is real at $w=0.1$ but shrinks sharply, and becomes negligible on single-step skill, at a much smaller weight.

Table 2: Geostrophic-weight sweep on the CNN, single seed (2026), $r=6$, vs. the seed-2026 CNN control (grid skill +0.3763, 20-day rollout skill -0.019).

Weight (λ_{geo})	Grid skill	Diff vs. control	20-day rollout skill	Diff vs. control
0.1 (DeepONet’s own, Table 1)	0.3161	-0.0602	-4.467	-4.447
0.01	0.3763	$+0.0000$	-0.412	-0.392
0.001	0.3792	$+0.0029$	-0.034	-0.014
0.0001	0.3782	$+0.0019$	-0.092	-0.073

This shows the harm is not a fixed property of “physics-informed constraints on the CNN” but a dose-response relationship: single-step skill cost drops from -0.0602 ($w=0.1$) to $+0.0000$ ($w=0.01$) to a statistically negligible $+0.0029$ / $+0.0019$ ($w=0.001$ / 0.0001 , both single-seed, within the noise band single-seed comparisons carry throughout this paper). Rollout stability follows the same pattern but does not fully recover at the smallest weights tested here: $w=0.01$ still costs real rollout stability (-0.412 vs. the control’s -0.019), while $w=0.001$ is close to the control (-0.034 , a -0.014 gap plausibly within single-seed noise) and $w=0.0001$ sits in between (-0.092 , a -0.073 gap). $w=0.001$ was the most promising single-seed candidate, and — mirroring exactly how this paper’s own learning-rate sweep was handled (single-seed screen, then 5-seed confirmation before a result was reported as a table) — we ran the 5-seed confirmation at that weight before treating the single-seed pattern as a finding.

Table 3: Geostrophic-consistency penalty at the CNN-appropriate weight ($w=0.001$) vs. the CNN control, seed-matched, $n=5$.

Seed	Grid skill			20-day rollout skill		
	Physics	Control	Diff	Physics	Control	Diff
2026	0.3766	0.3763	$+0.0003$	-0.064	-0.020	-0.045
7	0.3736	0.3747	-0.0011	-0.027	$+0.001$	-0.028
42	0.3964	0.3916	$+0.0048$	-0.078	-0.236	$+0.158$
123	0.3887	0.3697	$+0.0190$	-0.195	-0.191	-0.004
2027	0.3860	0.3860	$+0.0000$	-0.043	-0.026	-0.017
Mean			$+0.0046 \pm 0.0084$ ($t = 1.23$)			$+0.0127 \pm 0.0826$ ($t = 0.34$)

Confirmed conclusion: Table 1’s “architecture-dependent harm” finding was a **weight-selection artifact, not a property of the CNN**. At $w=0.001$, both grid-skill and 20-day rollout diffs are statistically indistinguishable from zero ($t = 1.23$ and $t = 0.34$, $n = 5$ — small next to Table 1’s own decisive $t = -21.66$ at $w=0.1$, or main text Sec. 3.6’s $t = +0.23$ null on AMSE’s own $r = 6$ grid-skill tie, which this paper already reports as “not distinguishable from noise” under the identical standard). The CNN was harmed in Table 1 specifically because it was tested at the DeepONet’s own weight, a hyperparameter tuned for a model with $33\times$ more parameters and an order of magnitude worse baseline skill. At a weight actually appropriate for the CNN, the same architecture Table 1 reported as significantly harmed shows no detectable cost on either metric.

The mechanistic account. The mechanism argument in main text Sec. 2.7 — that the residual constraint violation lives in the training labels, independent of which model reads them — is not wrong about the data, but it does not, by itself, predict whether a given model is *harmed* by

chasing a gradient toward a constraint the data does not exactly satisfy. A plausible mechanism, offered as a hypothesis consistent with the pattern and not one we tested directly (the same caution this paper applies to its own post hoc, unexplained rollout-stability observation at $r = 6$, main text Sec. 3.6): the constraint’s gradient is biased (it points toward the labels’ own residual violation, not toward ground truth), so its effect on a given model scales with how large that biased gradient is relative to the model’s own fit quality. At $w=0.1$, the biased signal is large enough to measurably compete with the data loss on a good model (the CNN) while barely registering against a poor model’s (the DeepONet’s) much larger baseline error; at $w=0.001$, the same biased signal is small enough, relative to the CNN’s tighter fit, that it no longer registers either — confirmed by Table 3’s 5-seed result, not merely hypothesized. This is the correct replacement for Sec. 2.7’s “architecture-agnostic, inert” claim, and for Table 1’s own “architecture-dependent, harmful” framing: neither architecture alone nor weight alone determines the outcome — it is weight and architecture jointly determining whether a biased gradient of a given size is large enough, relative to a specific model’s own error floor, to matter.

The corrected, general lesson from this paper’s physics-informed work is therefore narrower than either an “architecture-agnostic, always inert” claim or a clean “architecture-dependent, always harmful” one — and this paper reports the narrower, more qualified version specifically *because* it did the weight-appropriate check rather than stopping at Table 1: a soft physics-informed penalty’s effect — helpful, harmless, or harmful — depends on the *interaction* between the penalty’s weight and the base model’s own fit quality, not on architecture alone and not on the reanalysis’s residual noise floor alone. That the residual sits at the noise floor (Sec. 2.7) explains why the penalty carries little useful *signal* at any weight; it does not, on its own, predict at what weight the penalty stops being *safe* on a given model — that has to be checked per architecture, the same way learning rate does (S1 above), and now has been. We report the two-table result (Table 1’s real harm at an untuned weight, Table 3’s confirmed absence of harm at a tuned one) rather than the tidier single finding either simpler framing would have offered, consistent with this paper’s disclosure standard — and consider this the intended outcome of that standard, not a failure to reach a clean conclusion: checking one’s own hyperparameter-transfer assumption, per this paper’s own established practice, changed the answer.

References

- Lagerquist, R., & Ebert-Uphoff, I. (2022). Can we integrate spatial verification methods into neural network loss functions for atmospheric science? *Artificial Intelligence for the Earth Systems*, 1(4).
- Mason, E., Pascual, A., & McWilliams, J. C. (2014). A new sea surface height-based code for oceanic mesoscale eddy tracking. *Journal of Atmospheric and Oceanic Technology*, 31(5), 1181–1188.
- Roberts, N. M., & Lean, H. W. (2008). Scale-selective verification of rainfall accumulations from high-resolution forecasts of convective events. *Monthly Weather Review*, 136(1), 78–97.